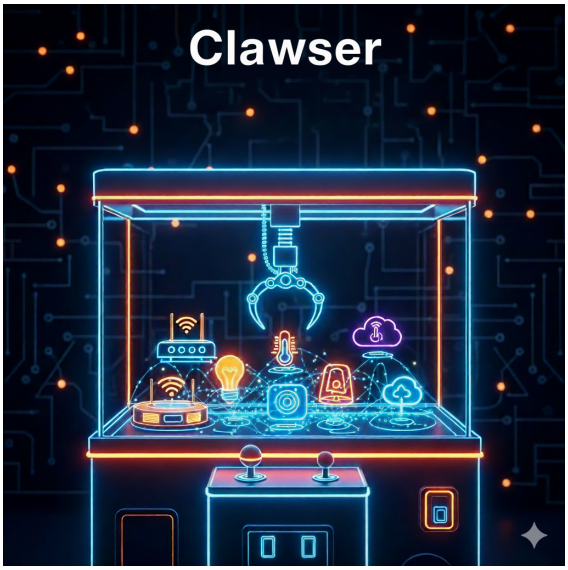
Department of Computer Science & Engineering
The University of Texas at Arlington

Architectural Design Specification
CSE 4316: Senior Design I
Fall 2025



Clawser
IOT Claw Game

Josalyn Abbott, David Aviles
Carson Brown, Phillip Cook
Justine Talia Gacho, Jeff Hernandez
Dang Khang Lam, Jesus Martinez
Aryan Prasain, Jordan Taylor
Jeff Wright, Alexander Zacarias

Revision History

Revision	Date	Author(s)	Description
0.1	10.22.2025	DA	document creation
0.2	11.02.2015	DA, JT, JA, AZ, PC, JTG, JH, DL, JM, CB, JW, AP	complete draft
1.0	05.01.2025	AZ, DA, PC, JA, JT, JTG, JH, DL, JM, CB, JW, AP	Final Submission

Contents

1 Introduction **6**

2 System Overview **7**
2.1 Cabinet Layer Description . . . 7
2.2 Cloud Layer Description . . . 7
2.3 Frontend Layer Description . . . 7

3 Subsystem Definitions & Data Flow **9**

4 Cabinet Layer Subsystems **10**
4.1 Power Supply . . . 10
4.2 E-Stop . . . 11
4.3 Cameras . . . 12
4.4 Computer . . . 13
4.5 Light Controller . . . 14
4.6 MCU . . . 15
4.7 External Sensors . . . 16
4.8 Gantry Motherboard . . . 17
4.9 Claw . . . 18

5 Cloud Layer Subsystems **20**
5.1 Websockets . . . 20
5.2 Stream Server . . . 21
5.3 Web Data Handler . . . 22
5.4 Database . . . 23
5.5 Play Handler . . . 24

6 Frontend Layer Subsystems **26**
6.1 Login Subsystem . . . 26
6.2 Create Profile Subsystem . . . 27
6.3 Dashboard Subsystem . . . 28
6.4 View Machines Subsystem . . . 30
6.5 In Claw State SubSystem . . . 32
6.6 Admin . . . 33
6.7 Buy Tokens . . . 35
6.8 Edit Profile . . . 36

List of Figures

1 A simple architectural layer diagram . . . 7
2 IOT Claw data flow diagram . . . 9
3 Power Supply subsystem description diagram . . . 10
4 E-Stop subsystem description diagram . . . 11
5 Cameras subsystem description diagram . . . 12
6 Computer subsystem description diagram . . . 13
7 Light Controller subsystem description diagram . . . 15
8 MCU subsystem description diagram . . . 16
9 External Sensors subsystem description diagram . . . 17
10 Gantry Motherboard subsystem description diagram . . . 18
11 Claw subsystem description diagram . . . 19
12 Websockets subsystem description diagram . . . 20
13 Stream Server Subsystem . . . 22
14 Web Data Handler subsystem . . . 23
15 Database subsystem description diagram . . . 24
16 Play Handler subsystem description diagram . . . 25
17 Login subsystem description diagram . . . 26
18 Create Profile subsystem description diagram . . . 27
19 Dashboard subsystem description diagram . . . 29
20 View Machines subsystem description diagram . . . 31
21 View Machines subsystem description diagram . . . 32
22 Admin subsystem description diagram . . . 34
23 Buy Tokens subsystem description diagram . . . 35
24 Edit Profile subsystem description diagram . . . 37

List of Tables

2 Power Supply interfaces . . . 11
3 E-Stop interfaces . . . 12
4 Cameras interfaces . . . 13
5 Computer interfaces . . . 14
6 Light Controller interfaces . . . 15
7 MCU interfaces . . . 16
8 External Sensors interfaces . . . 17
9 Gantry interfaces . . . 18
10 Claw interfaces . . . 19
11 Websockets interfaces . . . 21
12 Stream Server Subsystem interfaces . . . 21
13 Web Data Subsystem interfaces . . . 23
14 Database Subsystem interfaces . . . 24
15 Play Handler Subsystem interfaces . . . 25
16 Login Subsystem interfaces . . . 27
17 Create Profile Subsystem interfaces . . . 28
18 Dashboard Subsystem interfaces . . . 30
19 View Machines Subsystem interfaces . . . 31
20 In Claw State Subsystem interfaces . . . 33
21 Admin Subsystem interfaces . . . 35

22 Buy Tokens Subsystem interfaces 36
23 Edit Profile Subsystem interfaces 38

1 Introduction

Our product is an IoT-enabled claw machine system that integrates a mobile website, token management, and secure access features to deliver an interactive remote-control experience. The system combines hardware (cameras,a claw machine gantry, and a control board) with cloud-based software to support user authentication, game control, stream viewing, payment simulation, and administrative monitoring. The overall goal is to design a scalable and secure platform suitable for both entertainment and commercial environments. The primary purpose of the system is to allow users to remotely control a physical claw machine through a mobile interface, providing a real-time gaming experience. Users will sign in via a single sign-on (Google) mechanism, purchase or simulate tokens through a secure token system, and interact with the machine using intuitive controls. Administrators can access a diagnostic dashboard to monitor active machines, detect faults, and disable malfunctioning units. The system emphasizes standard secure practices, user accessibility, and responsive operation.

2 System Overview

The IoT Claw Machine system is organized into three primary architectural layers: the Cabinet Layer, the Cloud Layer, and the Frontend Layer. Each layer serves a distinct purpose within the overall architecture, enabling modular design, scalability, and maintainability. Communication between layers follows a clearly defined interface model:

- The Cabinet Layer communicates with the Cloud Layer.
- The Cloud Layer communicates with both the Cabinet Layer and the Frontend Layer.
- The Frontend Layer interfaces solely with the Cloud Layer.

This layered approach ensures that changes within one layer do not require major modifications to others, supporting flexibility and long-term evolution of the system.

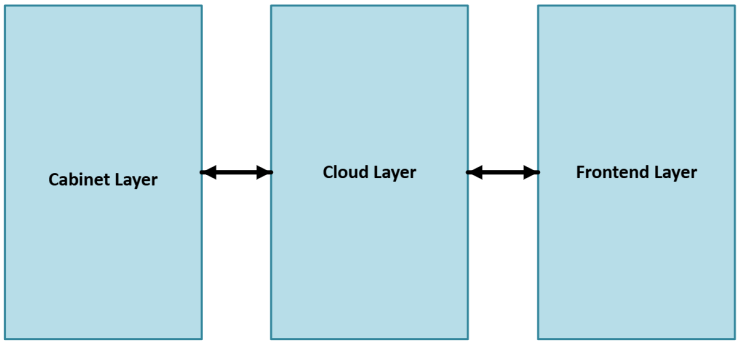


Figure 1: A simple architectural layer diagram

2.1 Cabinet Layer Description

The cabinet includes a power supply along with an E-Stop to power the internal cabinet mechanisms and provide a safe protocol to stop the machine under certain conditions. The power supply is used for the computer, the external sensors, the light controller, and gantry motherboard. The computer handles interactions between layers for the camera, stream data, and controls for the claw. The gantry motherboard controls claw strength and relays movement input to the claw.

2.2 Cloud Layer Description

The Cloud Layer serves as the intermediary between the frontend user and the hardware of the system. This layer is in charge of keeping track of basic user information such as their username, token balance, prize history, and JSON Google Authentication token. This layer is also where the gameplay will be handled using a play handler to keep track of players in the queue and who is currently playing or spectating the game. Another function of this layer is being in charge of the live video stream from the cabinet, ensuring the stream gets to the frontend for the user to view.

2.3 Frontend Layer Description

The Frontend Layer provides the user interface for the Clawser system, allowing users to interact with the IoT claw machine through a web browser. It orchestrates all user-facing functions, including login, account creation, machine selection, and live gameplay. This layer communicates directly with the Cloud Layer using secure HTTP and WebSocket connections to authenticate users, display machine status, transmit control inputs, and receive real-time video streams from the cabinet. The Frontend Layer

consists of multiple subsystems: the Login and Create Profile subsystems manage authentication and account setup, the Dashboard subsystem displays user information such as credits and recent activity, the View Machines subsystem shows available claw machines and queue information, and the In-Claw State subsystem handles real-time gameplay and video streaming.

3 Subsystem Definitions & Data Flow

Each subsystem is a logical unit responsible for a key function within its layer. The data flow between these subsystems and their interfaces with external entities (e.g., Google Authentication, HTTP clients, or user inputs) are illustrated below. The following diagram represents the detailed subsystem structure, showing both intra-layer and inter-layer data flows.

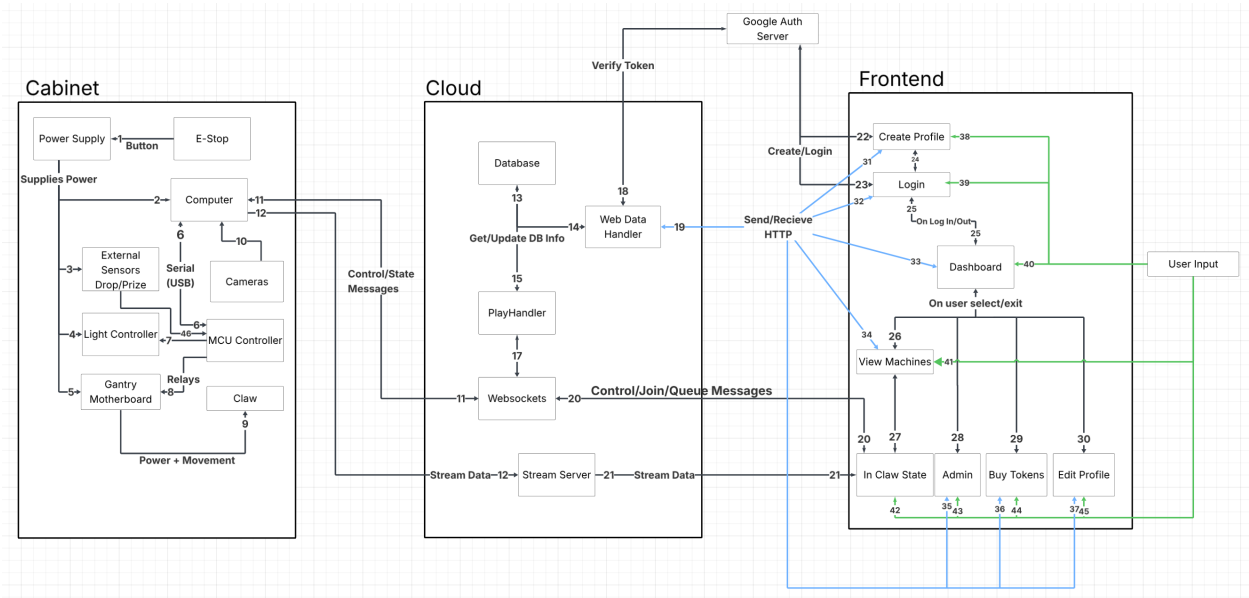


Figure 2: IOT Claw data flow diagram

4 Cabinet Layer Subsystems

The Cabinet Layer encompasses all physical and embedded components that make up the claw machine's operational hardware. It represents the lowest level of the IoT Claw Machine architecture and is responsible for real-time control, sensory feedback, and direct mechanical interaction with the environment. This layer interfaces with the Cloud Layer, receiving control commands and transmitting back video stream and cabinet status data.

The Cabinet Layer consists of several tightly integrated subsystems, each fulfilling a dedicated role. Together, these subsystems convert player actions into controlled movements of the claw mechanism, handle local feedback, and maintain reliable communication with the cloud and supervisory computer.

4.1 Power Supply

The power supply interacts with the computer, external sensors, light controller, and gantry motherboard. An E-Stop is used to disrupt this power supply with one press of a button for safety.

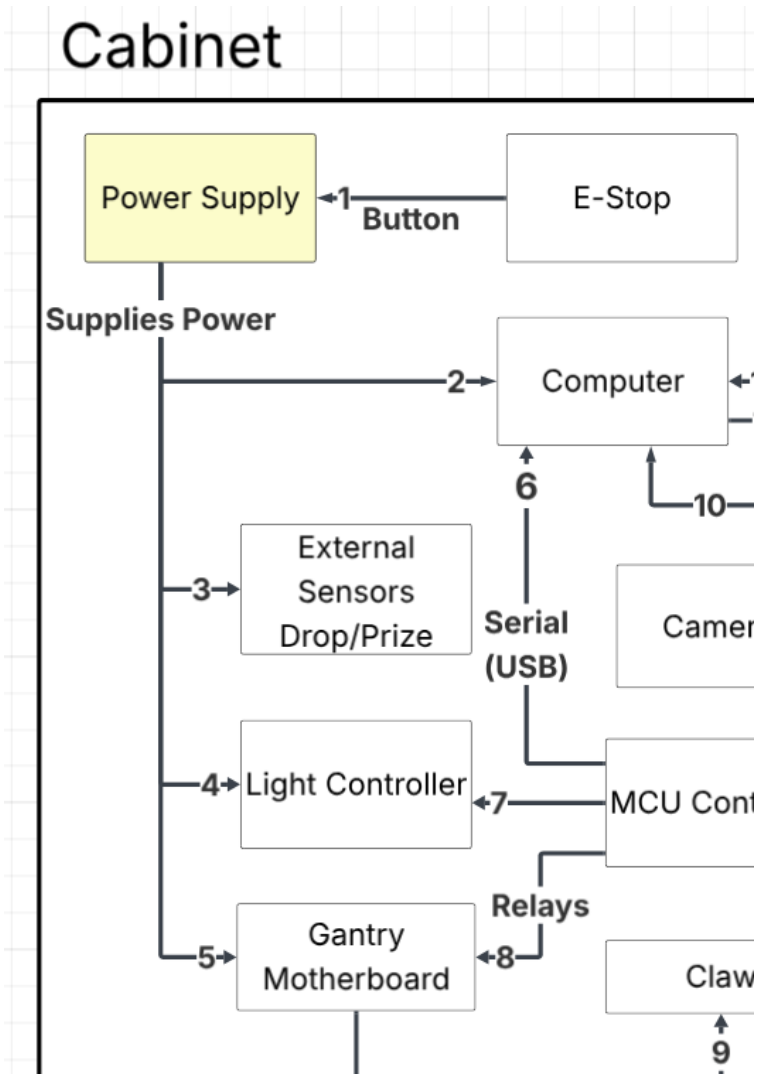


Figure 3: Power Supply subsystem description diagram

4.1.1 Assumptions

Power shall be provided to the other subsystems assuming the E-Stop button has not been pressed.

4.1.2 Responsibilities

The power supply provides power to the cabinet subsystems such as the computer, external sensors, light controller, and the gantry motherboard.

4.1.3 Subsystem Interfaces

Table 2: Power Supply interfaces

ID	Description	Inputs	Outputs
#1	E-Stop button interacts only with the power supply. If the button is pressed, power supply is turned off.	input 1: If the E-Stop is pressed, a signal is sent to cut off the power supply.	N/A
#2	The computer receives power from the power supply.	N/A	output 2: Power is sent to the computer.
#3	External sensors receive power from the supply to detect when a prize is dropped/won.	N/A	output 3: External sensors receive power from the power supply.
#4	Light controller receives power from the power supply	N/A	output 4: Power is supplied to the light controller.
#5	Gantry Motherboard receives power from the power supply.	N/A	output 5: Power is supplied to the gantry motherboard.

4.2 E-Stop

An E-Stop button is used to cut the power supply in cases of emergency or protocol.

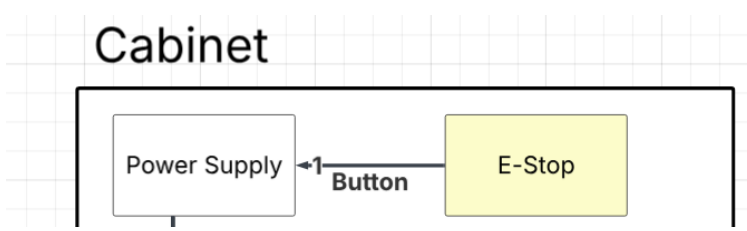


Figure 4: E-Stop subsystem description diagram

4.2.1 Assumptions

The emergency stop mechanism must fully disconnect the actuator control circuits within 1 second. The system reboots to default settings once power is restored.

4.2.2 Responsibilities

In the event of an emergency for whatever reason, the emergency stop cuts power to all the hardware components to stop any mechanical/electrical mishaps and allow maintenace people/person to work on the cabinet safely.

4.2.3 Subsystem Interfaces

Table 3: E-Stop interfaces

ID	Description	Inputs	Outputs
#1	If the E-Stop button is pressed, power will not be supplied to the cabinet.	N/A	output 1: The E-Stop button will cut off the power if pressed.

4.3 Cameras

Cameras provide live video direcly to the cabinet computer.

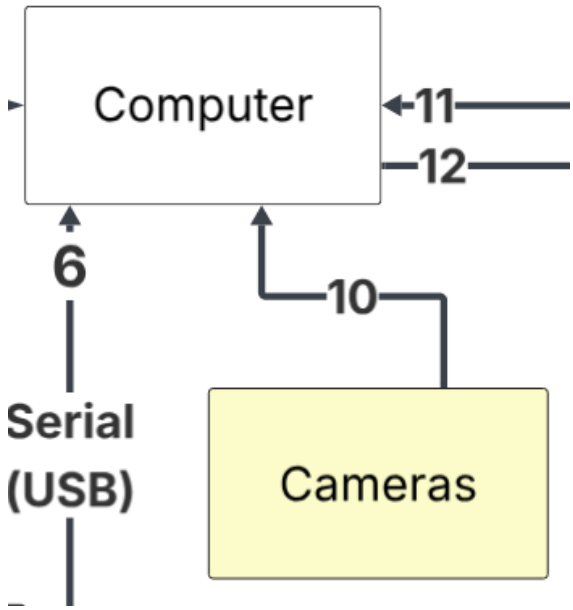


Figure 5: Cameras subsystem description diagram

4.3.1 Assumptions

It is assumed that all cameras are to remain on and sending video feed to the computer simultaneously regardless of which viewing angle the player uses.

4.3.2 Responsibilities

Cameras only interact with the computer, being powered on by it and sending video back to it. The computer uses that video feed and sends it as stream data to the cloud. There are two cameras mounted in different areas to provide different viewing angles to both the player and spectators. One is located on the inside of the cabinet looking down, the second one is mounted on the outside looking in.

4.3.3 Subsystem Interfaces

Table 4: Cameras interfaces

ID	Description	Inputs	Outputs
#10	Camera sends video feed to computer for use in video streaming.	N/A	output 10: Video data sent from Cameras to computer

4.4 Computer

The computer connects all the subsystems in this layer to communicate with other layers.

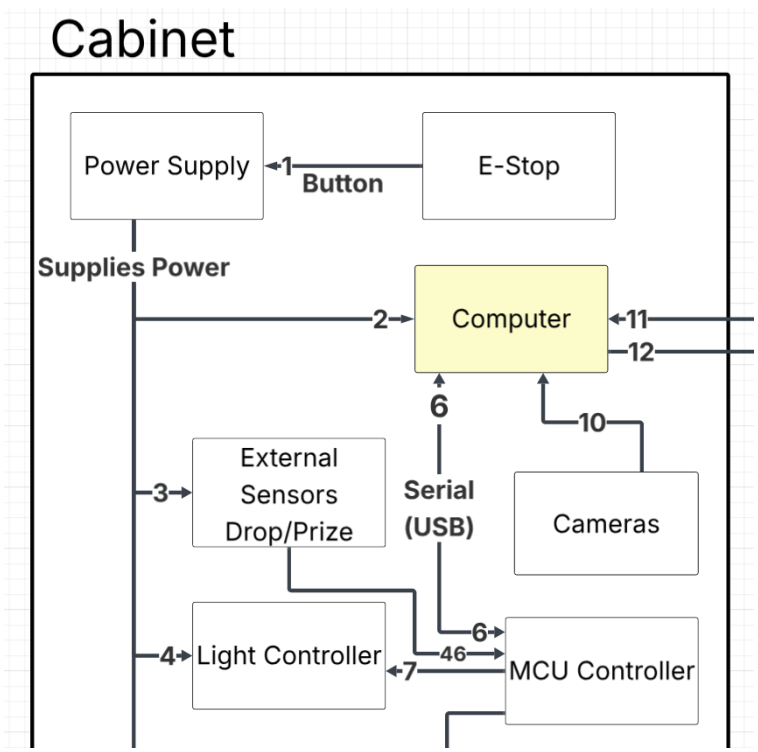


Figure 6: Computer subsystem description diagram

4.4.1 Assumptions

The computer should run a Microsoft Windows operating system or a Debian-based Linux distribution. It should also be assumed that the computer is properly connected to power, networked via a hardwired connection, and installed with the necessary software needed to operate.

4.4.2 Responsibilities

The cabinet computer serves as the hub subsystem for data transfer between the cabinet and the cloud layer. It receives input from the cameras and sends the data to the cloud by using software to convert the video to a viewable web stream. It also receives control data from the cloud to execute commands for physical outputs through the MCU and sends information/status data to the cloud regarding the physical cabinet.

4.4.3 Subsystem Interfaces

Table 5: Computer interfaces

ID	Description	Inputs	Outputs
#2	DC Power	input 2: DC voltage 12v	N/A
#6	Bi-Directional Serial over USB connection to the Microcontroller	input 6: Serial communications	output 6: Serial communications
#10	USB Connection of raw video data from cameras	input 10: Video data	N/A
#11	Bi-Directional Websocket data connection between the Cabinet and the Cloud	input 11	output 11
#12	Viewable Webstream output of camera feeds	N/A	output 12: Video data

4.5 Light Controller

The light controller allows for the control of lighting elements depending on events.

4.5.1 Assumptions

The lighting elements should be provided with their own source of power, along with the light controller, and should be managed by the MCU subsystem.

4.5.2 Responsibilities

The light controller provides lighting to the cabinet play area and allows for better visibily when viewed through the camera. Special lighting effects can be displayed depending on different events that can occur during gameplay.

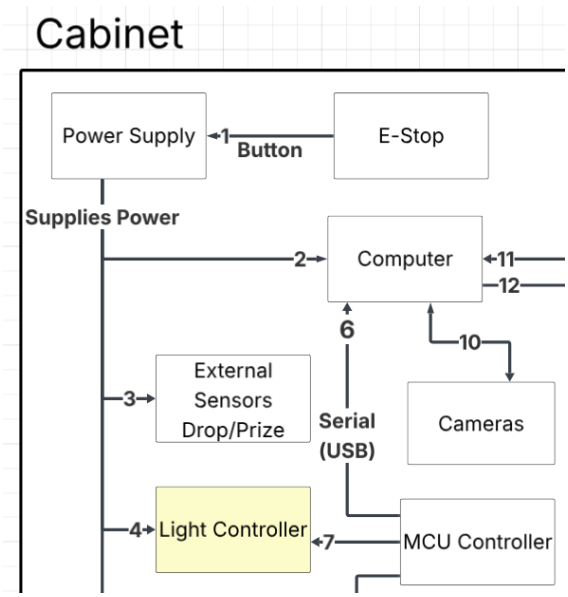


Figure 7: Light Controller subsystem description diagram

4.5.3 Subsystem Interfaces

Table 6: Light Controller interfaces

ID	Description	Inputs	Outputs
#4	Power Supply provides power for Light Controller	input 4: DC voltage 5-48v	N/A
#7	Signal from MCU Controller to change lighting	input 7: Digital control signals	Lighting

4.6 MCU

A Microcontroller Unit (MCU) connects the outside world and gantry kit to the computer.

4.6.1 Assumptions

The microcontroller will be programmed appropriately and provided with the correct electrical components to carry out its respective responsibilities. An agreed upon serial message system should be implemented both on the MCU and the computer to ensure proper communication.

4.6.2 Responsibilities

The MCU receives movement information from the computer and simulates the actions on the gantry motherboard through a relay system. The same relay system also simulates a coin insertion to start the game when ready. The MCU has an internal game clock and in conjunction with the external sensors, it will determine the outcome of the game and report it back to the computer. The MCU will adjust lighting accordingly depending on the game outcome. A raw control method will also be available to the computer for diagnostic purposes.

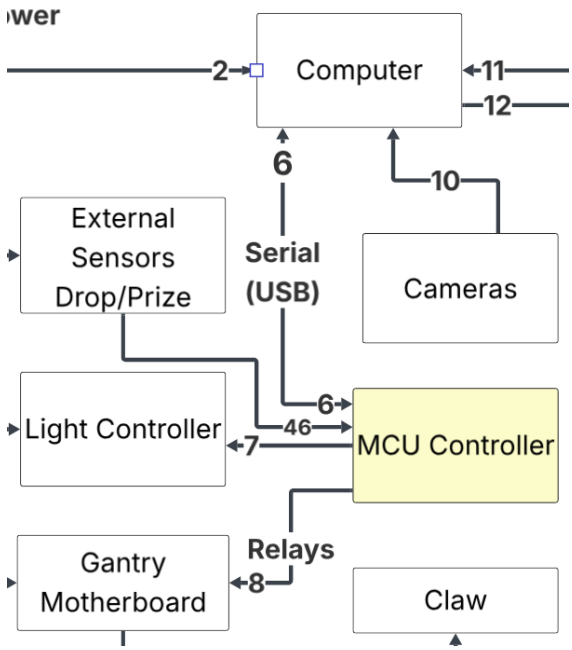


Figure 8: MCU subsystem description diagram

4.6.3 Subsystem Interfaces

Table 7: MCU interfaces

ID	Description	Inputs	Outputs
#6	Bi-Directional USB Serial communication to Computer	input 6: Serial communications	output 6: Serial communications
#7	Control signal to Light Controller to change lighting	N/A	output 7: Digital control signals
#8	Relay System to Gantry for Movement/Coin Insertion	N/A	output 8: Electrical control signals
#46	Signals from External Sensors	input 46: Analog/digital sensor inputs	N/A

4.7 External Sensors

External sensors capture events from the real-world and report it to the MCU.

4.7.1 Assumptions

A prize detection sensor is included in this subsystem that will communicate to the MCU. Some other sensors may be added to the subsystem that will require a power source.

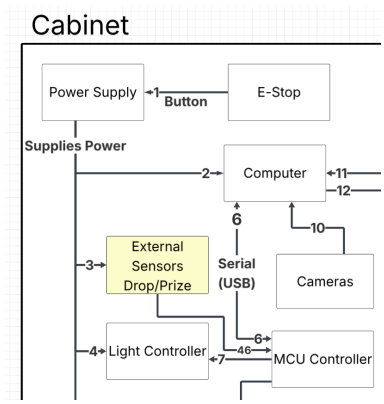


Figure 9: External Sensors subsystem description diagram

4.7.2 Responsibilities

At minimum, a prize sensor should detect when a user wins a prize and should report it to the MCU to take succeeding action. Other sensors can be included as safety checks to ensure proper operation of the cabinet layer.

4.7.3 Subsystem Interfaces

Table 8: External Sensors interfaces

ID	Description	Inputs	Outputs
#3	Power from Power Supply	input 3: DC voltage 12v	N/A
#46	Event signals from Sensors to report at the MCU	N/A	output 46: Analog/digital sensor outputs

4.8 Gantry Motherboard

The gantry motherboard controls the motors of the claw mechanism. This sends power to the claw and uses inputs to control the claw movement.

4.8.1 Assumptions

The gantry motherboard is from an already made component that comes with the claw gantry kit. A power supply, pins, and connectors for various inputs and outputs has been provided within the claw gantry kit.

4.8.2 Responsibilities

The gantry motherboard handles the electrical control of motors and the claw based on the input fed from the relay system of the MCU. It also has its own separate game logic that is replicated by the MCU and the computer. Based on the switches and dials on the board, a maintainer can set claw strength and timers that should be reflected in gameplay. The motherboard should also command the gantry to return to its starting position after every game and stop at the proper height for prize retrieval.

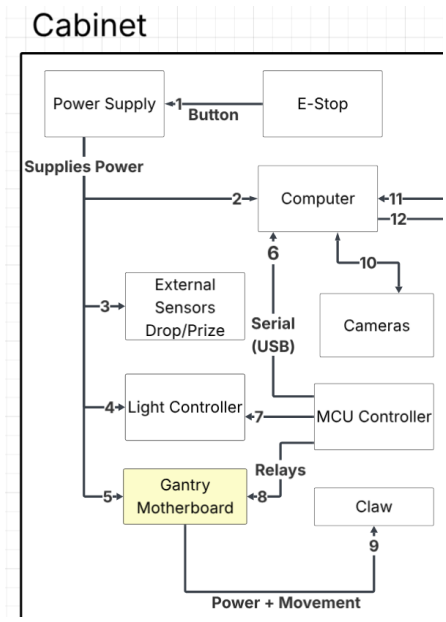


Figure 10: Gantry Motherboard subsystem description diagram

4.8.3 Subsystem Interfaces

Table 9: Gantry interfaces

ID	Description	Inputs	Outputs
#5	Power from Power Supply	input 5: DC voltage 48v	N/A
#8	Signal to gantry motherboard of received input	input 8: Electrical control signals	N/A
#9	Movement and action to claw	N/A	output 9: serial data

4.9 Claw

The claw is the physical component that is responsible for grasping and releasing prizes inside the claw machine cabinet.

4.9.1 Assumptions

The claw is only intended to move within the allowed x,y,z parameters specified by the computer. All prizes are within the claws weight limit.

4.9.2 Responsibilities

The claw performs the movements for grasping, lifting, and releasing prizes. It must do so according to real time control inputs from players while remaining within its position limits, as well as detect when it has picked up an item.

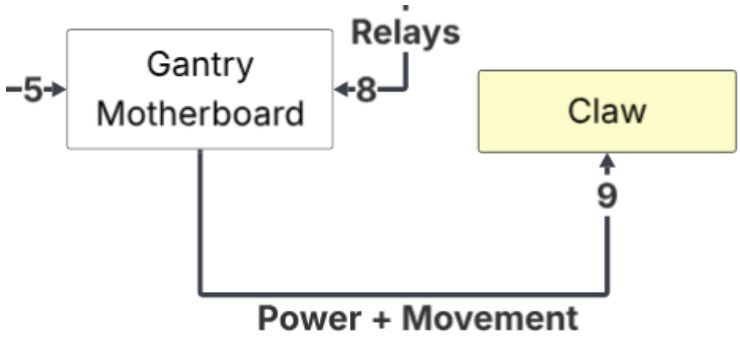


Figure 11: Claw subsystem description diagram

4.9.3 Subsystem Interfaces

Table 10: Claw interfaces

ID	Description	Inputs	Outputs
#9	Signal from motherboard to move / grab	input 9: serial data	N/A

5 Cloud Layer Subsystems

The Cloud Layer encapsulates the communication between the Cabinet Layer and the Frontend Layer. The Cloud Layer is responsible for keeping track of prize histories, player queues, play sessions, and ensuring that inputs from the users from the frontend are relayed to the Cabinet Layer so the claw machine can be moved without having physical access to the controls of the claw machine.

5.1 Websockets

The Websockets subsystem handles all commands sent to and from the hardware via websockets. This includes user commands to control the claw, switching the stream camera view, and machine state information. In addition, this subsystem communicates directly with the Play Handler, sending and recieving player queue information.

images/cloud/websockets.png

Figure 12: Websockets subsystem description diagram

5.1.1 Assumptions

This subsystem works under the assumption that both the frontend user and the microcontroller are capable of establishing and maintaing a persistent websocket connection. All communication via websockets use a predefined, structured format, such as JSON. It assumes that only the user currently playing is allowed to send commands to the claw which is properly handled by the play handler.

5.1.2 Responsibilities

The Websockets subsystem is primarily responsible for facilitating communication between the hardware and the frontend. Its main responsibility is real-time command handling, which involves receiving claw and stream control messages from the current player and sending it to the hardware computer. Any command input that does not come from the current active player should be dropped. Additionally, it handles the machine getting and sending the machines state and diagnostic information on a set

interval. This subsystem also must send queue join and queue leave commands that are received from the frontend to the play handler. In order to facilitate all of these functions it must perform connection management, establishing and terminating all websocket connections and handling connection errors.

5.1.3 Subsystem Interfaces

Table 11: Websockets interfaces

ID	Description	Inputs	Outputs
#11	Connection to hardware computer	Machine state Sensor/diagnostic data	Claw control commands Camera switch commands
#17	Connection to Play Handler	Player turn Queue information	Join/leave queue
#20	Connection to frontend	Claw control commands Join/leave queue Camera switch commands	output 1

5.2 Stream Server

The stream server is solely responsible for handling the stream data going from the hardware to the user. It serves as the intermediary between the cabinet layer and frontend layer ensuring the video stream is as low latency as possible for the use case of this project. This stream server takes care of any necessary encoding/compression needed for the user to be able to view the video feed on the frontend side.

5.2.1 Assumptions

This subsystem works under the assumption that the user's internet connection is fast enough to sustain a low latency video stream. This subsystem also assumes that the frontend layer will have the proper decoder and viewer to display the stream type being sent out by the stream server.

5.2.2 Responsibilities

The responsibilities of this subsystem are that of ensuring the video stream gets from the video capture hardware in the cabinet to the user side on the frontend. This layer is responsible for encoding the video to the proper type to be received by the frontend.

5.2.3 Subsystem Interfaces

Table 12: Stream Server Subsystem interfaces

ID	Description	Inputs	Outputs
#12	Live Video feed	Video Stream data	encoded video

images/cloud/stream_server.png

Figure 13: Stream Server Subsystem

5.3 Web Data Handler

The Web Data Handler subsystem serves as the interface between many subsystems in the frontend layer and the cloud, with communication between the two mainly involving user and machine data. When a user attempts to log in or create a profile, the Web Data Handler receives the user's authentication token from the frontend and forwards it to the Google Authentication Server for verification. Other than authentication, the Web Data Handler manages important user and machine data operations, it retrieves the claw machine's status and displays it as available in the user's list, it handles user token balances, and provides access to the user's play or prize history.

5.3.1 Assumptions

This subsystem works under the assumption that the user has a Google account to use to authenticate their account on our application. This handler also works with the assumption that the claw machine is capable of communicating with the cloud subsystem as a whole and is able to send the basic status messages.

5.3.2 Responsibilities

This subsystem is responsible for ensuring users on the application have a unique account each with their own token balance, username and play history. This subsystem is in charge of distinguishing between regular users and admin users and then granting the necessary permissions when they are an admin user. Another responsibility of this subsystem is ensuring that the frontend side is aware of the online status of the claw machine before attempting to join the queue.

images/cloud/web_data_handler.png

Figure 14: Web Data Handler subsystem

5.3.3 Subsystem Interfaces

Table 13: Web Data Subsystem interfaces

ID	Description	Inputs	Outputs
#14	Won prize/Machine status	JSON	N/A
#18	Google AUTH token	AUTH confirmation	JSON token
#19	User Input Data	JSON token HTTP messages	HTTP messages

5.4 Database

The Database subsystem is responsible for securely storing and managing all persistent data required by the system. It serves as the central repository for user information, gameplay records and some machine data. This subsystem operates locally on a Google Cloud Virtual Machine, providing availability, future scalability, and secure handling of data. Some of the things being stored here are account details (such as usernames and authentication identifiers), token balances, prize histories, and play session information.

5.4.1 Assumptions

The database assumes that the Web Data Handler is the only component with direct read/write access to the database, all other subsystems must route database requests through it. This implies that the Web Data Handler correctly validates all data before an insert or update and all sensitive information is

images/cloud/database.png

Figure 15: Database subsystem description diagram

properly handled (hashed) before being stored. In addition it is assumed that a well-defined database schema is in place and that the database itself is not publicly accessible.

5.4.2 Responsibilities

The database subsystem's primary responsibility is user data management. This involes securely storing all user account details, including profile data, current token balance, and win/play history. The database will provide a query interface via the web data handler to retrieve this data for login, or update it, such as decrementing tokens after a game or adding a new user. Another major responsibility is gameplay and session logging, serving as the systems ledger by storing records for all play sessions, transactions, and wins. Additionally, the subsystem is responsible for machine data storage, maintaining a list of all claw machines, including their configuration and ID.

5.4.3 Subsystem Interfaces

Table 14: Database Subsystem interfaces

ID	Description	Inputs	Outputs
#13	Connection to Web Data Handler	SQL commands	N/A

5.5 Play Handler

The Play Handler subsystem is responsible for managing the player queue. It serves as a medium to keep players' track in the queue and if they are allowed to get in "line." This allows for the security that only

the main player has access to the cameras that are only given to the actual player who is controlling the claw machine. Furthermore, it helps separate what players are doing in each claw machine: spectating, queued, or actively playing. It also checks if the player has ended the game and won a prize.

images/cloud/play_handler.png

Figure 16: Play Handler subsystem description diagram

5.5.1 Assumptions

The Play Handler subsystem assumes that the Web Data Handler is providing correct, up-to-date information on a player's status. It also categorizes an "active player" as a user who is currently playing with the claw machine, "queue" as the line of users waiting to play the claw machine and "spectator" as an user who is not in the queue. Both the "spectator" and "queue" users should not have the ability to change input from the claw machine.

5.5.2 Responsibilities

The Play Handler subsystem's main responsibility is keeping track of who is the active player controlling the claw machine while keeping track of users in a queue. This includes checking with the Web Data Handler subsystem to ensure that users meet the requirements to join a queue, if there is an user playing it, or play the claw machine. Additionally it will check for inputs received from the Websockets subsystem to verify and allow the claw machine to be moved according to the player. Other responsibilities include that queued and spectating players do not have the ability to change the input of the claws.

5.5.3 Subsystem Interfaces

Table 15: Play Handler Subsystem interfaces

ID	Description	Inputs	Outputs
#15	Receiving and Sending Information	JSON token	JSON token
#17	Sending User Input	JSON token	JSON token

6 Frontend Layer Subsystems

The Frontend Layer allows users to view and control the IoT claw machine via a GUI interface. It provides a login subsystem for third-party authentication, a subsystem for profile creation for new users, and a dashboard for logged-in users to manage their tokens and interface with available claw machines. There are also subsystems for allowing a user to control a claw, allowing an administrator access to privileged actions, and for users to purchase tokens for play sessions.

6.1 Login Subsystem

The Login Subsystem handles displaying a GUI for the user signing in and sends required data to both Google's servers and our own Backend Server for verification.

images/Frontend/login.png

Figure 17: Login subsystem description diagram

6.1.1 Assumptions

As the application currently only supports Google accounts, the user might not be able to sign in if Google authentication servers were to go offline. Similarly, this would also be true if our backend server goes offline.

6.1.2 Responsibilities

The Login Subsystem displays the widgets required for the user to sign in. When the user presses sign in with Google, it prompts the user to sign in to their Google account. Once the login is successful and Google server sends the app a JSON Web Token (JWT), we then send this to the Backend Layer for verification. If the Backend Layer's response is successful, then we ass the user's data to the Dashboard Subsystem. If it was unsuccessful, an appropriate error message is displayed.

6.1.3 Subsystem Interfaces

Table 16: Login Subsystem interfaces

ID	Description	Inputs	Outputs
#39	User Inputted Google Account Info	Google Account Info	N/A
#23	Communication with Google's Auth Server	Google Account Info	HTTP Status code JSON Web Token
#32	Communication with Backend Server	JSON Web Token	HTTP Status code JSON User Data
#25	Dashboard Subsystem Change	JSON User Data	N/A

6.2 Create Profile Subsystem

This subsystem is responsible for allowing new users to register and create an account in the Clawser platform. It is part of the Frontend Layer, and it communicates with the Login subsystem and the Cloud Auth and Database services. Its primary function is collecting user details, validating required consent, and initiating the creation of a user profile in the backend. Once created, the subsystem transitions the user flow to the Dashboard

images/Frontend/create_profile.png

Figure 18: Create Profile subsystem description diagram

6.2.1 Assumptions

This subsystem operates under the assumption that the user has not yet created an account and therefore does not possess valid authentication credentials. The platform expects that each user will have a Google Account and their servers and subsequently ours will be online. Communication between the Create Profile interface and backend components takes place over secure, encrypted channels to protect sensitive personal data such as passwords and identity tokens. It also assumes that backend services will

complete credential verification, and account creation and storage. Assumes that any failure or denial from backend services will be handled gracefully and communicated back so the user may correct input issues or try again later.

6.2.2 Responsibilities

The primary responsibility of this subsystem is to collect all necessary user-entered information for account creation, including email address, password for OAuth identity, display name, and legal consent. It must validate that the information provided by the user satisfies required formats and policies before transmitting the data to backend services. Once validation is complete, the subsystem must initiate the account creation transaction by communicating with the Authentication and Data Handler services. Its responsibilities continue after registration, as it must respond appropriately to the outcomes of backend processing. If registration is successful, the subsystem either starts a logged-in session automatically or transitions to the login state. If registration fails due to issues such as invalid data or a duplicate email, the subsystem must clearly communicate these issues and allow the user to correct them.

6.2.3 Subsystem Interfaces

Table 17: Create Profile Subsystem interfaces

ID	Description	Inputs	Outputs
#22	Communication to Google Auth Server	User Google account Info	JSON Token
#38	User Inputting Account Information	User details	User Information
#31	Verification/ Account Creation	JSON token and Account info	Confirmation response that activates the account and provides session authentication
#24	UI navigation to Login or Dashboard	Successful creation event	Transition to Login or Dashboard

6.3 Dashboard Subsystem

The Dashboard subsystem serves as the primary home screen for authenticated users after logging into the Clawser platform. Its role is to provide an organized interface where users can monitor their personal status and navigate to different system features. It displays important account information such as available credits, recent play results, and profile status. From this view, users can select a specific machine to observe or join, initiate the process of purchasing credits, or navigate to administrative features if their permissions allow it. The subsystem continuously reflects real-time updates from the cloud regarding machine availability and state transitions, ensuring that users always make decisions based on the most current data in the system.

6.3.1 Assumption

The Dashboard subsystem assumes that the user is already logged in and holds a valid authenticated session. It also assumes that credit information is accurate. Furthermore, the system assumes that users have appropriate role assignments that determine whether the Dashboard displays general user

images/Frontend/dashboard.PNG

Figure 19: Dashboard subsystem description diagram

functionality or if they can click and view privileged admin tools. Finally, it assumes that network connectivity remains stable enough for the UI to reflect state changes in a timely manner and provide fallback messaging if connectivity is disrupted.

6.3.2 Responsibilities

The Dashboard subsystem is responsible for presenting a clear and intuitive overview of the user's account. It must request and display data from the backend, such as the number of credits the user has available. It enables the user to navigate to the credit purchase function, modifying user settings, accessing administrative controls when permitted, or to view connected machines. In addition, it must gracefully handle communication or backend service failures by notifying the user and attempting to refresh the displayed information. The Dashboard therefore acts as both a navigational hub and a real-time status monitor within the user interface architecture.

6.3.3 Subsystem Interfaces

Table 18: Dashboard Subsystem interfaces

ID	Description	Inputs	Outputs
#40	User Input	User input can click buttons to transition states	State change
#25	User hits log out	User Input	State change to the login screen
#26	User hits view machines	User Input	State change to the view machines state
#28	User hits Admin	User Input	State change to the admin state
#29	User hits Buy Tokens/Shop	User Input	State change to the shop screen
#30	User hits edit profile	User Input	State change to the profile edit screen
#33	Dashboard requesting a HTTP response for info	HTTP GET request	HTTP Response

6.4 View Machines Subsystem

The View Machines subsystem is the interactive gateway through which users browse and select available claw machines to observe or join for gameplay. It sits between the Dashboard and the In-Game State subsystems within the Frontend layer, serving as a real-time window into the status of machines currently connected to the cloud infrastructure. Its primary purpose is to display machine metadata such as name, location identifier, queue length, current player state, and connection health. When a user chooses a machine, the subsystem initiates a join request through the cloud, waits for authorization from the Play Manager, and transitions the interface to the In-Game State view once control is granted.

6.4.1 Assumption

The View Machines subsystem operates under the assumption that the user has an active authenticated session to request access to a machine. Only available and healthy machines should be selectable options. The subsystem assumes a stable network connection between the client and the Cloud backend to maintain accurate status data. It also assumes that authorization tokens passed to the Play Manager during join requests will be validated before the session is established.

6.4.2 Responsibilites

The View Machines subsystem is responsible for presenting a dynamic list of machines that a user can observe or attempt to join. It retrieves initial machine data from the backend database. It must display key details for each machine such as its identifier, stream preview, queue position, and whether it is currently occupied and allow users to filter or sort this list for easy selection. When the user selects a machine, a request is made to the server to ensure its not full. If successful, the subsystem change to

images/Frontend/view_machines.PNG

Figure 20: View Machines subsystem description diagram

the In Claw Subsystem State. If the join request is denied or times out, the View Machines subsystem must inform the user and return to a safe, idle display state.

6.4.3 Subsystem Interfaces

Table 19: View Machines Subsystem interfaces

ID	Description	Inputs	Outputs
#34	State requesting machine info	HTTP GET Request	HTTP Response with JSON info about machines
#41	User Input	User Input	State change
#26	Change state to dashboard	User input hit button	Change back to the dashboard state.
#27	Change state to the in claw state	User selected a machine	Transition to In-Game view for stream and control session initialization.

6.5 In Claw State SubSystem

The In-Game State subsystem governs the live, interactive experience during a player's active session with a claw machine. It begins operating once the Play Manager grants control via Websocket Communications to a user who has joined a machine through the View Machines subsystem. Within this state, the user can view the live video feed from the cabinet, issue movement and claw commands, join the queue, and monitor the countdown timer that limits each session. The subsystem recieves and displays stream video data from the Stream Receiver, control acknowledgments and state notifications from the WebSocket connection communicating to the Cloud Play Manager to create a synchronized, low latency game interface. Its core function is to ensure that every player input results in an accurate, immediate physical response from the machine while preserving fairness, timing, and safety constraints. When the session ends, the subsystem presents the outcome, including prize detection results.

images/Frontend/view_machines.PNG

Figure 21: View Machines subsystem description diagram

6.5.1 Assumption

The In-Game State subsystem assumes that the user is fully authenticated and has been authorized by the Cloud Play Manager to take control of a specific claw machine. It presumes that only one active control session exists per machine at any given time and that all other users observing the same machine are in a read only viewing state. The subsystem relies on a stable WebSocket connection for transmitting player commands and receiving immediate feedback. It also assumes continuous streaming from the machines cameras, with latency low enough to allow responsive movement control. Power, lighting, and safety systems inside the cabinet are assumed to be already active and ready to receive external instructions. In case of network disruption or backend timeout, the subsystem assumes that the Cloud layer will reclaim control gracefully and reset the machine to an idle or attract mode, ensuring the next player can safely begin a new session.

6.5.2 Responsibilities

This subsystem is responsible for orchestrating all live gameplay interactions between the user and the physical claw machine. It must capture directional input and claw action commands from the user interface, encode them into lightweight network messages, and deliver them reliably to the Cloud Play Manager through the WebSocket service. It also has the responsibility of receiving acknowledgment messages or error signals, translating these into corresponding visual updates on the user interface, and maintaining synchronization with the live video feed. During gameplay, the subsystem manages a countdown timer, provides real time status indicators such as connection health and remaining credits, and visually displays success or failure outcomes once the claw releases. Overall, it ensures the user perceives a continuous, responsive, and immersive gaming experience tightly coupled with the physical machines behavior.

6.5.3 Subsystem Interfaces

Table 20: In Claw State Subsystem interfaces

ID	Description	Inputs	Outputs
#20	Communication interface between Frontend and Cloud Play Manager via WebSocket	User control commands and queue information	Acknowledgments and machine state updates reflected on UI
#21	Live video stream interface from Stream Receiver to Frontend player view	Continuous Whep/WebRTC encoded camera feed data	Rendered video display
#27	Exit back to View Machine State	User hitting back button	Return to view machine state
#42	User Input	User Input	Changing state or sending websocket messages

6.6 Admin

The Admin subsystem provides privileged users the ability to oversee, manage, configure, and temporarily disable all connected claw machines within the Clawser platform. Functioning as the operational control panel, it offers an interface for monitoring live telemetry, toggling machine states, scheduling maintenance, viewing usage analytics, and diagnosing connection or hardware faults. The subsystem communicates primarily with the Cloud Auth service for authorization and with the Data Handler and Play Manager services for retrieving and modifying machine data.

6.6.1 Assumption

The Admin subsystem assumes that users accessing it have already been authenticated and verified as administrators through the Cloud Auth service. It presumes that all administrative actions such as enabling or disabling machines, changing configurations, or viewing sensitive analytics are executed only within a secure HTTPS session and accompanied by valid role based tokens.

images/Frontend/admin.PNG

Figure 22: Admin subsystem description diagram

6.6.2 Responsibilities

The Admin subsystem is responsible for giving administrators a comprehensive view of the platforms operational landscape and for enabling safe, controlled interventions when required. Its first responsibility is to display a unified dashboard summarizing each machines current state, including whether it is online, in play, idle, or under maintenance. It must allow authorized personnel to send commands that toggle machine availability, reset connections, clear queues, or modify configuration parameters such as session duration or claw strength limits. It also provides access to detailed machine telemetry, historical play data, and user activity summaries for diagnostic and performance analysis purposes. Beyond active control, the subsystem supports reporting capabilities by compiling daily or weekly summaries for management review.

6.6.3 Subsystem Interface

Table 21: Admin Subsystem interfaces

ID	Description	Inputs	Outputs
#43	User Interaction with the admin screen	User input	HTTP Requests/State change
#35	Communication to backend server	Admin commands or Requesting Admin info	Acknowledgment messages in HTTP or JSON data
#28	Transition back to dashboard	User hit back button	State Change back to the dashboard

6.7 Buy Tokens

The Buy Tokens subsystem provides the mechanism through which authenticated users can purchase digital tokens or credits that grant them access to play claw machine sessions on the Clawser platform. It acts as the bridge between the financial gateway and the users in-app account balance. When a user initiates a purchase, this subsystem communicates with the Cloud Data Handler and external payment processor APIs to authorize transactions, record payment confirmations, and update the corresponding credit balance stored in the platform database.

images/Frontend/buy_tokens.PNG

Figure 23: Buy Tokens subsystem description diagram

6.7.1 Assumption

This subsystem assumes that the user is already logged in and authenticated with a valid, active session token issued by the Cloud Auth service. It presumes that users possess an associated payment method

either stored credentials through a trusted payment gateway or temporary authorization via a secure checkout session. All communication between the frontend, the backend Data Handler, and the external payment processor occurs over encrypted HTTPS channels to protect sensitive information.

6.7.2 Responsibilities

The Buy Tokens subsystem is responsible for guiding the user through every step of the purchase workflow, from package selection to confirmation of payment and credit update. It presents multiple token packages with corresponding prices, validates the user's intent, and ensures that payment authorization is completed through the designated external service. Upon receiving a payment confirmation, the subsystem updates the displayed credit balance in real time by communicating with the Cloud Data Handler, which adjusts the user's account record in the backend database. In addition to handling immediate transactions, it also maintains a log of purchase history accessible to the user through the Dashboard for transparency.

6.7.3 Subsystem Interface

Table 22: Buy Tokens Subsystem interfaces

ID	Description	Inputs	Outputs
#44	User Interacting With The Screen	User input	HTTP Requests/State change
#36	Communication to backend server	HTTP Get Request	Acknowledgment messages in HTTP or JSON data
#29	Transition to dashboard	User hit back button	State Change back to the dashboard

6.8 Edit Profile

The Edit Profile subsystem allows authenticated users to view and modify their personal account information within the Clawser platform. It serves as a controlled environment where users can update details such as display name, avatar image, password, or contact email while preserving the integrity of their identity and stored preferences. Operating as part of the Frontend layer, the subsystem communicates with the Cloud Auth and Data Handler services to validate submitted changes, verify user credentials when sensitive fields are altered, and commit updates to the backend database.

6.8.1 Assumption

The Edit Profile subsystem assumes that the user is already logged in with a valid authenticated session provided by the Cloud Auth service. It presumes that all update requests originate from verified users who have previously completed account creation and, if applicable, email verification. We are also assuming the server is online.

6.8.2 Responsibilities

The Edit Profile subsystem is responsible for providing users with a cohesive interface for managing their personal information while preserving data integrity. Its first responsibility is to retrieve and display the

images/Frontend/edit_profile.PNG

Figure 24: Edit Profile subsystem description diagram

current profile information from the backend database when the user accesses the profile page. It must then allow the user to modify editable fields, perform immediate validation on input, and highlight any violations of format rules before submission. Once changes are confirmed, the subsystem packages the updated data into a secure request and transmits it to the Data Handler service. Upon receiving confirmation from the backend, it updates the interface to reflect the new data and provides visual feedback that the modification was successful.

6.8.3 Subsystem Interface

Table 23: Edit Profile Subsystem interfaces

ID	Description	Inputs	Outputs
#45	User Interacting With The Screen	User input	HTTP Requests, Display Changes, State Change
#37	Communication to backend server	HTTP GET/POST Requests	Acknowledgment messages/responses in HTTP and/or JSON data
#30	Transition back to dashboard	User hit back button	State Change back to the dashboard